Instrukcja manualnego badania funkcjonalności TestGEN (golden data)

Spis treści

TestGEN — instrukcja manualnego badania funkcjonalności (GUI + golden data)

0. Przygotowanie — jednorazowe (≈ 3 minuty)
1. Analiza dokumentacji (wymagania, klasyfikacja, niespójności)
2. OCR skanu PDF (ścieżka wymagająca przetwarzania obrazu)
3. Analiza kodu → przypadki testowe (AST)
4. Generatory kodu testów (codegen)
5. Analiza UML (modele, sekwencje, XMI, wizualne)
6. Analiza GUI — zrzuty ekranu i HTML/DOM
7. Self-healing — regresja wizualna (pary zrzutów)
8. Bezpieczeństwo (SAST, taint, konfiguracja, auto-patch)
9. Luki funkcjonalne — dokumentacja vs kod
10. Raporty i eksport
11. Integracje — konfiguracja i import z Jiry
12. Rozliczenia (opcjonalnie; środowisko testowe płatności)

Co zrobić, gdy coś nie działa (bez dostępu do serwera)

Zestawienie scenariuszy i plików golden (ściąga)

TestGEN — instrukcja manualnego badania funkcjonalności (GUI + golden data)

> **Dla kogo:** tester / użytkownik TestGEN, który ma dostęp do aplikacji (przeglądarka) oraz do **repozytorium kodu** (stąd pobierze pliki testowe „golden"), ale **NIE ma** dostępu do serwera, uprawnień administratora i nie uruchamia żadnych skryptów. Cała praca odbywa się klikaniem w GUI. Zakładamy, że jesteś już zalogowany/a (tworzenie konta pominięte).

Jak czytać tę instrukcję

- Elementy interfejsu zapisane tak, jak widzisz je w aplikacji: przycisk — **„Utwórz"**, ekran — *Dokumenty*, pole — *Nazwa*.
- Ścieżki plików są względne do głównego katalogu repozytorium TestGEN, np. `backend/tests/golden_data/docs/brd/brd_platnosci.docx`.
- Przy każdym scenariuszu jest sekcja **Ground truth** — plik `..._ground_truth.json` z katalogu golden zawiera **oczekiwane wyniki**

> (liczby, treści). Otwierasz go obok aplikacji i porównujesz 1:1.

- „Na co czekać" opisuje widoczne w GUI stany pośrednie (spinnery, statusy) — to normalna część przepływu, nie błąd.
- Czasów nie podajemy co do sekundy: małe pliki kończą się zwykle w kilka–kilkanaście sekund, duże dokumenty i archiwa — do minuty i dłużej.

---

0. Przygotowanie — jednorazowe (≈ 3 minuty)

0.1. Utwórz pusty projekt do testów

1. W górnym menu kliknij **„Projekty"**.
2. Kliknij przycisk **„Nowy projekt"** (prawy górny róg listy).
3. W oknie dialogowym wypełnij:
   - *Nazwa* — wpisz `Testy manualne golden`,
   - *Opis (opcjonalnie)* — możesz pominąć,
   - *Język (opcjonalnie)* — zostaw puste (scenariusze i tak są wielojęzyczne).
4. Kliknij **„Utwórz"**. Projekt pojawi się na liście jako pusty (0 plików).
5. Wejdź w projekt — zobaczysz jego kartę (dashboard) z grupami przycisków.

0.2. Zasada routingowa — ekran decyduje, nie plik

Ten sam plik może być różnie potraktowany w zależności od tego, **przez który ekran go wgrasz** (np. PDF = dokument *albo* diagram UML). Pięć ekranów wgrywania, wszystkie z karty projektu:

Przycisk na karcie projektu	Przyjmuje rozszerzenia
„Wgraj pliki" (kod źródłowy)	`.py .java .js .ts .cs .php .kt …` + archiwa `.zip`, `.tar.gz`
„Dokumenty"	`.pdf .docx .md .html .txt`
„Diagramy UML"	`.puml .xmi .xml .svg .pdf .png .jpg`
„Zrzuty ekranu GUI"	`.png .jpg .jpeg .bmp .tif .webp`
ekran GUI z HTML/DOM (z ekranu zrzutów)	`.html .htm` (+ opcjonalny `a11y.json`)

Nie wgrasz nigdzie: `.sql`, `.csv`, `.yaml`, `.json` (wyjątek: `a11y.json` na ekranie DOM), `.xlsx`. Pliki `*_ground_truth.json` służą wyłącznie do porównania wyników — **nie wgrywamy ich**.

0.3. Wspólne elementy każdego wgrywania

1. Każdy ekran wgrywania ma strefę „Przeciągnij pliki tutaj lub kliknij, aby wybrać" — możesz przeciągnąć pliki lub kliknąć i wybrać je z dysku.
2. Pod strefą widać listę wrzucanych plików z paskiem postępu.
3. Po zakończeniu przy pliku pojawia się plakietka **„OK"**; porażka — **„Błąd"** z krótkim powodem przy pliku; anulowanie — **„Anulowano"**.
4. Jeśli wgrasz plik o tej samej nazwie i rozmiarze co istniejący w projekcie, aplikacja zapyta: „Zamienić czy anulować?" — wybierz **„Zamień"** albo **„Anuluj"**.

0.4. Limity i blokady — zanim zaczniesz

- **Liczba analiz jest limitowana planem** (np. Basic: 10 analiz w okresie miesięcznym). Jedna uruchomiona analiza może objąć wiele plików naraz — **najpierw wgraj wszystko, co chcesz przeanalizować, i dopiero potem uruchom jedną analizę**; nie odpalaj analizy po każdym pojedynczym pliku.
- Niektóre moduły mogą wymagać wyższego planu: ich przyciski na karcie projektu są **wyszarzone z kłódką**; najechanie pokazuje podpowiedź „Funkcja dostępna w planie …". Kliknięcie prowadzi na stronę planów (subskrypcji) — to celowe zachowanie, nie błąd.
- Gdy skończy się limit analiz, przy próbie uruchomienia zobaczysz komunikat o limicie (licznik resetuje się z nowym okresem rozliczeniowym albo wymaga wyższego planu).

---

1. Analiza dokumentacji (wymagania, klasyfikacja, niespójności)

Cel: z BRD (dokument Word z celowo wprowadzonymi problemami) dostać klasyfikację dokumentu, listę wymagań i wykryte niespójności.

Pliki (ekran *Dokumenty*):

- `backend/tests/golden_data/docs/brd/brd_platnosci.docx`

Ground truth:
`backend/tests/golden_data/docs/brd/brd_platnosci_ground_truth.json` — spodziewaj się m.in.: typ dokumentu `specyfikacja_wymagań` (pewność co najmniej 50%), **co najmniej 7 wymagań** zaczynających się od `WYM-001: System musi obsługiwać płatności jednorazowe kartą.`, itd.

Kroki:

1. Na karcie projektu kliknij **„Dokumenty"**.
2. Kliknij strefę wgrywania i wskaż `brd_platnosci.docx`; poczekaj na **„OK"**.
3. **Na co czekać:** dokument przechodzi statusy **„Przetwarzanie"** → **„Gotowy"** na liście dokumentów (parsowanie działa w tle). Dopóki widnieje „Przetwarzanie", wyniki będą niekompletne — odczekaj.
4. Kliknij nazwę dokumentu — otworzy się podgląd:
   - karta **„Sekcje"** — pełna struktura rozdziałów z nawigacją po lewej (każdy nagłówek dokumentu = klikalna sekcja);
   - lista sekcji po lewej pozwala wyszukiwać (*Szukaj sekcji…*).
5. Wróć na kartę dokumentu i wejdź w **„Wymagania"**:
   - każde wymaganie ma identyfikator (`WYM-…`), typ (Funkcjonalne/ Niefunkcjonalne), priorytet, pewność (%) oraz pasek *Sekcja źródłowa* z nazwą rozdziału i przyciskiem **„Podgląd"**;
   - kliknij **„Podgląd"** przy wymaganie — wracasz do podglądu dokumentu **z podświetloną na żółto sekcją źródłową** i przewinięciem do niej;
   - filtry u góry (*Typ*, *Priorytet*, szukajka) zawężają listę.
6. Wejdź w **„Niespójności"** — dokument ma **celowe** problemy (duplikaty, brakujące odniesienia): każda niespójność to rozwijany panel z opisem, powiązanymi wymaganiami i sugestią; status możesz zmienić przyciskami „Zmień status" (Otwórz / Akceptuj / False positive / Naprawione).
7. Wejdź w **„Glosariusz"** — terminy z definicjami, liczbą wystąpień i typem (Dziedzina/Techniczny/Biznesowy/Akronim); możesz edytować definicję (**„Edytuj definicję"**) i oznaczyć termin jako zweryfikowany (**„Zweryfikuj"**).

Weryfikacja (odhacz):

- ☐ status dokumentu „Gotowy",
- ☐ klasyfikacja zgodna z ground truth (typ + pewność ≥ progu),
- ☐ liczba wymagań ≥ 7; treści `WYM-001…` zgadzają się z ground truth,
- ☐ co najmniej jedna niespójność z kategorii celowych,
- ☐ „Podgląd" wymagań podświetla właściwą sekcję.

2. OCR skanu PDF (ścieżka wymagająca przetwarzania obrazu)

Cel: PDF będący skanem (bez warstwy tekstu) — tekst wyekstrahowany OCR.

Plik (ekran *Dokumenty*):
`backend/tests/golden_data/docs/srs/srs_system_logowania.pdf`

Ground truth: `docs/srs/srs_system_logowania_ground_truth.json`.

Kroki:

1. Ekran **„Dokumenty"** → wgraj plik → poczekaj na „OK" i status „Gotowy".
2. **Na co czekać:** OCR trwa wyraźnie dłużej niż zwykłe parsowanie — status „Przetwarzanie" może wisieć od kilkunastu sekund do minuty.
3. Otwórz podgląd dokumentu → karta „Sekcje": powinny być rozdziały wyekstrahowane ze skanu.

Możliwa sytuacja — środowisko bez przetwarzania obrazu: jeśli skan nie zostanie odczytany, dokument skończy ze statusem **„Błąd"** albo podgląd będzie pusty. To ograniczenie środowiska, nie Twoje działanie — odnotuj i pomiń scenariusz, zgłaszając wynik razem z nazwą pliku.

Weryfikacja: liczba sekcji/wymagań zgodna z ground truth.

---

3. Analiza kodu → przypadki testowe (AST)

Cel: z pliku źródłowego wygenerować przypadki testowe (unit).

Plik (ekran *Wgraj pliki*):
`backend/tests/golden_data/code_samples/python/auth_handler.py`

Kroki:

1. Na karcie projektu kliknij **„Wgraj pliki"** → wgraj `auth_handler.py` → „OK".
2. Wejdź w **„Konfiguracja analizy"**:
   - **Parametry analizy** — zaznaczone moduły: zostaw *Analiza kodu* i *Generowanie testów*;
   - **Framework testowy** — na razie **„— Automatyczny —"**;
   - kliknij **„Analizuj"**.
3. **Na co czekać:** pasek postępu z etapami **Kolejkowanie** → **Parsowanie** → **Analiza** → **Generowanie testów** → **Finalizacja**. Nie zamykaj karty; po „Finalizacji" aplikacja sama przejdzie do wyników.
4. Wyniki analizy kodu: struktury logiczne (klasy/funkcje), punkty wejścia, zależności, przepływ danych.
5. Wejdź w **„Przypadki testowe"**: tabela z kolumnami Tytuł / Typ / Priorytet / Status; użyj „Grupuj według" (Moduł / Priorytet / Typ).

Weryfikacja: dla `auth_handler.py` przypadki testowe obejmują klasy i funkcje z pliku; każda karta przypadku ma metadane (Framework, Język, Plik źródłowy, Funkcja) oraz zakładkę „Kod testu".

3a. Auto-detekcja frameworka (bardziej zaawansowane)

Framework wykrywany jest z plików konfiguracyjnych wgranych **razem z kodem**. Przygotuj obok pliku `.py` własny `requirements.txt` zawierający jedną linijkę `unittest` (albo wgrywając kod Java — `pom.xml` z zależnością `org.testng:testng`; dla C# — plik `.csproj` z pakietem MSTest). Po analizie w metadanych przypadków zobacz framework `unittest` / `testng` / `mstest` — a nie domyślny dla języka. Jawny wybór frameworka w „Konfiguracji analizy" **nadpisuje** detekcję.

3b. Inne języki

Wgraj kolejno (każdy osobnym scenariuszem, ekran *Wgraj pliki*): `code_samples/java/…`, `code_samples/csharp/…`, `backend/tests/fixtures/golden/php/LaravelController.php` (starsza warstwa fixture — kontroler Laravela z traits; sprawdź, że struktury są rozpoznane).

3c. Martwy kod

W wynikach analizy kodu znajdź sekcję wykrycia martwego kodu; wyniki można triage'ować (oznaczenie fałszywego trafienia). Użyj własnego pliku z nieużywaną funkcją lub przykładu z `code_samples` — sprawdź, że element oznaczony jako martwy ma widoczne uzasadnienie.

---

4. Generatory kodu testów (codegen)

Cel: z zaznaczonych przypadków wygenerować plik testowy w wybranym frameworku i porównać z oczekiwanym wyjściem golden.

Przygotowanie: wykonaj najpierw scenariusz 3 (są przypadki testowe).

Ground truth (do porównania 1:1):
`backend/tests/golden_data/codegen/unittest/expected_output.txt` (oraz `testng/`, `mstest/`) — dla próbek Calculator z `code_samples`.

Kroki:

1. Z poziomu *Przypadków testowych* przejdź do **„Scenariusze Gherkin"**, a stamtąd kliknij link **„Generator kodu testowego"**.
2. Zaznacz co najmniej jeden przypadek (brak zaznaczenia → komunikat „Wybierz co najmniej jeden przypadek testowy.").
3. Wybierz **Framework** (8 opcji: Cypress, SikuliX, Selenium, Playwright, Appium, unittest, TestNG, MSTest).
4. Opcje warunkowe: dla Cypress — *Bazowy URL*; dla Appium — *Platforma urządzenia* (Android—UiAutomator2 / iOS—XCUITest).

5. Kliknij **„Generuj kod {nazwa}"** → komunikat „Generowanie uruchomione (zadanie: {identyfikator})". **Zapisz ten identyfikator (UUID).**
6. **Na co czekać:** w sekcji „Pobierz wygenerowany kod" wklej UUID → kliknij **„Pobierz kod"**; jeśli generowanie jeszcze trwa, odczekaj chwilę i spróbuj ponownie.
7. Kliknij właściwy przycisk pobrania (`.py` / `.cy.ts` / `.spec.ts` / `.java` / `.cs`) — plik zapisze się na dysku.

Weryfikacja per framework (odhacz po pobraniu):

- ☐ Cypress `.cy.ts` — komendy `cy.visit` / `cy.get` + asercje,
- ☐ Selenium `.py` — Selenium WebDriver,
- ☐ Playwright `.spec.ts` — import `@playwright/test`,
- ☐ Appium `.py` — `AppiumBy` + capabilities zgodne z wybraną platformą,
- ☐ unittest `.py` — klasa dziedzicząca po `unittest.TestCase` (`self.assertEqual…`),
- ☐ TestNG `.java` — adnotacje `@Test` / `@BeforeMethod`,
- ☐ MSTest `.cs` — `[TestClass]` / `[TestMethod]`, `Assert.AreEqual`,
- ☐ SikuliX `.py` — wzorce obrazowe wycięte z bbox.

Generatory specjalistyczne (osobne strony, linki ze strony Gherkin):

- „Generator kodu testów bazodanowych" — przypadki typu database,
- „Generator kodu testów PHP" — po wgraniu `code_samples/php/Calculator.py` → `Calculator.php`; przełącznik PHPUnit/Pest.

Porównanie golden: wgraj próbki Calculator (`code_samples/python`, `java`, `csharp`), wygeneruj kod w unittest/TestNG/MSTest i porównaj pobrane pliki z `codegen/*/expected_output.txt`.

---

5. Analiza UML (modele, sekwencje, XMI, wizualne)

Pliki (ekran *Diagramy UML*):

Co badasz	Plik	Ground truth — kluczowe liczby
Klasy/dziedziczenie	`uml/class/class_inheritance.puml`	**4 elementy, 3 relacje**
Sekwencja z fragmentami	`uml/sequence/sequence_alt.puml`	bloki alt/loop/opt w przepływie
Model z narzędzia CASE	`uml/class/xmi_class_model.xmi`	ten sam kształt modelu co z PlantUML
Diagram wizualny (SVG)	`uml/visual/class_order.svg`	ekstrakcja bez tekstu źródłowego
Wiele diagramów naraz	`uml/cross_diagram/…`	user flows spięte z różnych diagramów

Kroki (wzór dla `class_inheritance.puml`):

1. Karta projektu → **„Diagramy UML"** → wgraj plik → „OK".
2. **Na co czekać:** parsowanie diagramu w tle (status przy pliku); po zakończeniu diagram jest klikalny.
3. Kliknij diagram → widok klasy: klasy z atrybutami/metodami, linie dziedziczenia/interfejsów/kompozycji.
4. Dla sekwencji: widok przepływu wywołań z blokami alternatyw/pętli.
5. Z ekranu UML kliknij **„Analizuj user flows"** (dla zestawów cross-diagram) → wygenerowane przepływy; z przepływu możesz przejść do **Gherkin** (scenariusze BDD).

Weryfikacja: liczby elementów/relacji z tabeli zgodne z widokiem; dla XMI — struktura identyczna z odpowiednikiem `.puml`.

---

6. Analiza GUI — zrzuty ekranu i HTML/DOM

Pliki (ekran *Zrzuty ekranu GUI*):

Co badasz	Plik	Ground truth
Formularz logowania	`gui/forms/login_form.png`	**5 komponent** inputy, przycisk
Dashboard z KPI	`gui/dashboards/analytics_dashboard.png`	karty/wykresy/ rozpoznane
GUI z HTML + audyt dostępności	`backend/tests/fixtures/golden/dom/login_form.html` (+ `a11y.json`)	komponenty + audytu WCAG

Kroki:

1. Karta projektu → **„Zrzuty ekranu GUI"** → wgraj PNG → „OK".
2. **Na co czekać:** detekcja komponentów (obraz!) trwa dłużej niż parsowanie tekstu; status pliku zmienia się na gotowość po zakończeniu.
3. Kliknij zrzut → wynik: inwentarz komponentów (input/button/link/form) z ramkami zaznaczenia (bbox) i pewnością rozpoznania.
4. Wariant DOM: na ekranie GUI przejdź do trybu **HTML/DOM** → wgraj `login_form.html` wraz z `a11y.json` → komponenty + audyt dostępności.

Weryfikacja: liczba komponentów dla `login_form.png` = 5 (patrz `gui/forms/login_form_ground_truth.json`: formularz „Logowanie", pola, przycisk, link odzyskiwania).

---

7. Self-healing — regresja wizualna (pary zrzutów)

Cel: dwie wersje tego samego ekranu → klasyfikacja zmiany (MINOR_DIFF = drobna, auto-naprawa; MAJOR_DIFF = krytyczna).

Pliki (ekran *Zrzuty ekranu GUI*, wgraj parami):

Para	Zmiana	Oczekiwany st
`visual_regression/pair_form_removed_baseline.png` + `…_comparison.png`	usunięty kontener	**MAJOR_DIFF**
`visual_regression/pair_element_added_baseline.png` + `…_comparison.png`	dodany element	**MINOR_DIFF**
`visual_regression/pair_button_moved_*`	przesunięty przycisk	MINOR_DIFF
pozostałe pary `pair_*`	—	status z plików `pair_*_groun`

Kroki:

1. Wgraj **oba pliki pary** na ekran zrzutów GUI (baseline i comparison).
2. Uruchom analizę GUI (jak w scenariuszu 6, jeśli wymaga analizy).
3. Wynik porównania: status pary + wizualne zaznaczenie różnic.

Weryfikacja: statusy zgodne z tabelą; MAJOR_DIFF wyraźnie odróżnialny od MINOR_DIFF w wyniku.

---

8. Bezpieczeństwo (SAST, taint, konfiguracja, auto-patch)

Pliki (ekran *Wgraj pliki* — kod i pliki konfiguracyjne wgraj jako kod):

Co badasz	Plik	Oczekiwany rezultat
Wstrzyknięcie SQL (CWE-89)	`security/vulnerable_code/sqli/sqli_python.py`	podatność wykryta kategorią CWE-89 i lokalizacją
Błędna konfiguracja	`security/config/Dockerfile`, `security/config/deployment.yaml`	wyniki misconfig (secrets/uprawnieni
Auto-patch	`security/patches/sqli_login.py` (i porównawczo `sqli_login_fixed.py`)	automatyczna propozycja łatki

Kroki:

1. Wgraj pliki na ekranie **„Wgraj pliki"** (kod razem z plikami konfiguracyjnymi).
2. Na karcie projektu wejdź w moduł **„Bezpieczeństwo"** i uruchom skan (jeśli przycisk ma kłódkę — patrz 0.4).
3. **Na co czekać:** etapy analizy jak w scenariuszu 3.
4. Wyniki: lista znalezisk z priorytetami (krytyczne/wysokie/…), kategorią, plikiem i linijką; przefiltruj po kategorii.
5. Dla SQLi otwórz znalezisko → **„Rekomendacje"**: opis i propozycja łatki; dla `sqli_login` możesz porównać proponowaną łatkę z plikiem `sqli_login_fixed.py` z repo.

Weryfikacja: SQLi wykryte jako krytyczne/wysokie; łatka zmienia niebezpieczne zapytanie na parametryzowane (zgodnie z `_fixed.py`).

---

9. Luki funkcjonalne — dokumentacja vs kod

Cel: wymagania z dokumentu porównane z kodem → brakujące funkcje, procent pokrycia.

Przygotowanie: wgraj i przeanalizuj **oba** źródła w jednym projekcie: dokument (scenariusz 1: `brd_platnosci.docx`) oraz kod (scenariusz 3: `code_samples/python/auth_handler.py`), a następnie uruchom analizę obejmującą oba zakresy.

Kroki:

1. Na karcie projektu wejdź w **„Analiza pokrycia"** (moduł zgodności dokumentacja↔kod).
2. **Na co czekać:** porównanie wymagań z bytami kodu — postęp jak przy innych analizach.
3. Wynik: procent pokrycia, lista wymagań ze statusem pokrycia (zaimplementowane / częściowe / brak), mapowanie na encje kodu, sekcja luk krytycznych.

Weryfikacja: wymagania płatności nie mają pokrycia w `auth_handler.py` (logowanie ≠ płatności) — statusy „brak" z uzasadnieniem; procent pokrycia spójny z liczniami na karcie podsumowania.

---

10. Raporty i eksport

Cel: wygenerować raport z wykonanych analiz i wyeksportować przypadki testowe do formatu TMS.

Kroki:

1. Karta projektu → **„Raporty"** → wybierz zakres (projekt / moduł / przypadek) i format (PDF / HTML / JSON) → **„Generuj raport"**.
2. **Na co czekać:** raport powstaje chwilę; po gotowości pobierz plik o nazwie `testgen_{projekt}_{data}_{format}`.
3. Z listy **„Przypadki testowe"** zaznacz przypadki (możesz zaznaczyć wiele) → **„Eksportuj"** → dialog „Eksport do TMS":
   - format: TestRail CSV / Jira Xray XML / JSON,
   - zakres eksportu,
   - „Mapowanie pól" — przycisk „Przywróć domyślne" przywraca domyślne mapowanie.
4. Pobierz plik i otwórz (CSV w arkuszu, XML/JSON w edytorze).

Weryfikacja: raport zawiera wyniki z wcześniejszych scenariuszy; eksport zawiera wyłącznie zaznaczone przypadki z kompletem pól.

11. Integracje — konfiguracja i import z Jiry

Cel: podłączyć Jirę/Azure DevOps i zaimportować wymagania z projektu Jiry (razem z załącznikami).

Kroki (Jira):

1. Menu → **Integracje** (Ustawienia) → wybierz Jira → wypełnij adres instancji, e-mail i token API → zapisz. Status integracji zmieni się na „Połączono".
2. Karta projektu → **„Import z Jiry"**:
   - wybierz projekt Jiry z listy (lista pobiera się z Twojej instancji),
   - typy zgłoszeń: zostaw domyślne (Story/Epic/Bug/Sub-task),
   - kliknij **Importuj**.
3. **Na co czekać:** import działa w tle i pokazuje postęp procentowy (pobieranie zgłoszeń → zapis wymagań → pobieranie załączników → analiza luk). Przy kilkudziesięciu zgłoszeniach z załącznikami licz na minuty.
4. Po zakończeniu: status importu „Ukończono" z licznikami (wymagania / załączniki / pominięte). Zaimportowane wymagania zobaczysz w *Wymaganiach* w dokumencie `jira-{KLUCZ}.md`; **załączniki-dokumenty (np. PDF) dostają własne parsowanie** — na liście dokumentów przechodzą statusy „Przetwarzanie → Gotowy" tak jak wgrane ręcznie.
5. Drugi import z tego samego projektu Jiry = tryb przyrostowy (nowe/zmienione zgłoszenia od ostatniego importu).

Weryfikacja: liczniki importu > 0; wymagania widoczne na liście; załącznik-PDF osiąga „Gotowy" i ma sekcje w podglądzie.

---

12. Rozliczenia (opcjonalnie; środowisko testowe płatności)

Kroki: menu → „Plany i zakup" (trzy plany: Freemium 0 / Basic 30 / Premium 50 PLN miesięcznie), „Subskrypcja" (zmiana planu), „Faktury" (dane fakturowe, pobranie faktury po płatności), „Zużycie i powiadomienia" (postępy 80/90/100% limitów). Płatność idzie przez środowisko testowe bramki — karta testowa z dokumentacji bramki; po płatności subskrypcja aktywna, faktura dostępna.

Weryfikacja: liczniki zużycia rosną po analizach ze scenariuszy 1–9; przekroczenie progu pokazuje komunikat limitu (opisany w 0.4).

Co zrobić, gdy coś nie działa (bez dostępu do serwera)

1. **Status „Błąd" przy pliku** — odczytaj powód przy pliku (za duży plik, nieobsługiwany format). Sformatuj poprawnie i wgraj ponownie.
2. **Analiza wisi na etapie** dłużej niż kilka minut — odśwież stronę; jeśli etap się utrzymuje, zgłoś test z: nazwy projektu, nazwy pliku, godziny uruchomienia i ostatniego widocznego etapu.

3. **Kłódka na funkcji** — wymagany wyższy plan (patrz 0.4); to nie błąd.
4. **Komunikat o limicie** — patrz 0.4; nie da się go obejść z poziomu GUI.
5. Wyniki testów notuj w tabeli: scenariusz / data / wynik (zgodne z ground truth? TAK/NIE) / uwagi — ten sam format wystarczy do zgłoszenia.

Zestawienie scenariuszy i plików golden (ściąga)

#	Funkcjonalność	Plik golden (ekran wgrywania)
1	Analiza dokumentacji	`docs/brd/brd_platnosci.docx` (*Dokumenty*)
2	OCR PDF	`docs/srs/srs_system_logowania.pdf` (*Dokumenty*)
3	Analiza kodu → testy	`code_samples/python/auth_handler.py` (*Wgraj pliki*)
4	Generatory kodu testów	wynik scen. 3 + `codegen/*/expected_output.txt`
5	UML	`uml/class/class_inheritance.puml` i in. (*Diagramy UML*)
6	Analiza GUI / DOM	`gui/forms/login_form.png`, `fixtures/golden/dom/login_form.html`
7	Self-healing	`visual_regression/pair_*_*.png` parami
8	Bezpieczeństwo	`security/vulnerable_code/sqli/sqli_python.py`, `security/config/*`
9	Luki funkcjonalne	scen. 1 + scen. 3 w jednym projekcie
10	Raporty/eksport	wyniki scen. 1–9
11	Integracje/import Jiry	własna instancja Jiry
12	Rozliczenia	środowisko testowe płatności

Wszystkie ścieżki względem `backend/tests/golden_data/` (starsze fixtury w `backend/tests/fixtures/golden/` — wskazane explicite tam, gdzie użyte). Szczegóły ekranów: podręcznik użytkownika (części 1–5).